Mobile App Security Cheat Sheet

Hossam Atef

20 October 2026

The whole book in a 20-minute read. Use it the night before an interview, a design review, or an exam. Each idea is in plain words first, then what to say. If an answer feels thin, follow the § link to the section that explains it.

Current to October 2026: Android 17, iOS 27, MASVS v2.1.0, MASWE v1.0.0, MASTG v2.0.0. Printable PDF: `dist/Mobile-Security-Cheat-Sheet.pdf`.

---

1. The one idea

Your app runs on a device you don't control. The person holding the phone can read your code, watch and change your traffic, change what your functions return, and read your storage and memory. Almost every good answer in mobile security follows from this. Almost every bad design forgets it.

Five rules come out of it:

1. **Nothing in the app is secret.** Keys, client secrets, and AI system prompts can all be read. Design so that a found secret is useless.
2. **The server decides; the app only sends signals.** Prices, user identity, permissions, and limits are checked on the server, every time.
3. **Keep keys in secure hardware.** Both platforms can create a key inside secure hardware and use it there, so the key never enters your app's memory.
4. **Trust the platform defaults.** The sandbox, storage encryption, and strict HTTPS are on by default. Many real bugs are a team switching one off.
5. **Detect and report; don't block on the phone.** A check on the phone can be removed. Send it to the server as a signal and let the server decide.

---

2. Words you must know (one line each)

Basics (Chapters 0.1 and 0.5)

Term	Plain meaning
Confidentiality, integrity, availability (CIA)	Keep it secret, keep it unchanged, keep it working
Authentication vs authorization	*Who are you?* vs *What are you allowed to do?* They fail separately
Threat model	A structured answer to "what could go wrong with this feature, and what do we do about it?"
Trust boundary	A line where data passes from something you control to something you don't. Check everything that crosses it
Attack surface	Every place an attacker can send input to your app
Defense in depth	Several layers, so one failure isn't a breach. Layers count only if they fail independently
Risk	Likelihood × impact. Rank by risk, not by how scary a bug sounds

Cryptography (Chapter 0.2)

Term	Plain meaning
Symmetric encryption	One shared key locks and unlocks (AES)
Asymmetric encryption	A public key and a private key; signs and agrees on keys (elliptic curves, RSA)
Hash	A fingerprint of data. One way: you can't get the data back (SHA-256)
MAC (HMAC)	A hash with a secret key, so only key holders can make or check it
Digital signature	Proof made with a private key that anyone with the public key can check
AEAD (AES-GCM)	Encryption that also detects tampering. Your default
Nonce	A number used once per message. Never reuse one with the same AES-GCM key
Password hashing	Slow on purpose, on the server: Argon2id or scrypt

Sign-in (Chapter 0.3)

Term	Plain meaning
OAuth 2.0	The standard way an app gets permission to call an API for a user
PKCE	A one-time secret that proves the app that started sign-in is the one finishing it. Use `S256`
Public client	An app that can't keep a secret. Every mobile app is one
Access token	Short-lived key to call the API. Keep it in memory
Refresh token	Long-lived key to get new access tokens. Your most valuable credential
DPoP	Ties a token to a key on the device, so a stolen token is useless elsewhere (§12.6)
Passkey	Passwordless sign-in: the device holds a private key per site and unlocks it with a biometric or the device PIN (§11.7)

Platform and hardware (Chapter 0.4, Chapter 4)

Term	Plain meaning
Sandbox	Each app gets its own private storage that other apps can't read
Android Keystore	The system that keeps your keys in secure hardware; your app holds only a name for the key
TEE	A protected area inside the main processor where keys are used
StrongBox	An optional separate security chip on some Android phones. Slower, stronger
Secure Enclave	Apple's separate secure processor. Its keys never leave it unencrypted
Keychain	iOS storage for small secrets. Its *accessibility class* decides when an item can be read
Root / jailbreak	Removing the platform's restrictions so the owner (or attacker) controls everything
Frida	A tool that injects code into a running app and changes what functions return

Network (Chapters 7–8)

Term	Plain meaning
TLS	Encryption and server identity for network traffic (HTTPS)
Certificate authority (CA)	An organization your phone trusts to vouch for server identities
Certificate pinning	Your app accepts only specific keys for your server, not any trusted CA's
SPKI pin	A hash of the server's public key. Survives renewals that keep the same key; the right thing to pin
Certificate Transparency (CT)	Public logs of every publicly trusted certificate, so you can spot one you didn't request

Proving who is calling (Chapters 9–12)

Term	Plain meaning
Play Integrity	Google tells *your server* whether the request came from your real app on a real device
App Attest	Apple's version: a Secure Enclave key proves requests come from your real app
Key attestation	Proof, signed by the hardware, of where a key lives and what protects it (Chapter 5)
Backend-for-Frontend (BFF)	Your own thin server that holds secrets and makes trust decisions for the app
Risk score	Combine many signals into a graded response (allow, step up, limit, block) instead of one yes/no

Standards (Chapter 3)

Term	Plain meaning
OWASP Mobile Top 10	An awareness list of common risks. You can't test against it
MASVS	The standard: 24 controls saying *what must be true*
MASWE	78 weaknesses saying *what goes wrong*
MASTG	The testing guide: *how to check*. Cite only tests `MASTG-TEST-0200` and above
MAS profiles	L1: baseline for every app. L2: high-risk apps; assumes the OS may be rooted or jailbroken. R: resilience, the user is the adversary (always on top of L1 or L2). P: privacy. Say "the MAS-L2 profile", not "MASVS L2"
STRIDE	Six threat types: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege (Chapter 0.5)

---

3. Every topic in a few lines

Storage (Chapter 6)

- Android: keep the access token in memory. Store the refresh token encrypted with Tink, a Keystore key, and DataStore (`datastore-tink`). `EncryptedSharedPreferences` is deprecated.
- iOS: Keychain with `kSecAttrAccessibleWhenUnlockedThisDeviceOnly`. For background work, `AfterFirstUnlockThisDeviceOnly`. Never `Always`.
- Plain storage inside the sandbox isn't the usual problem. **Leaks** are: logs, backups, screenshots, the keyboard, notifications, and the clipboard (§6.5).

Network (Chapters 7–8)

- The defaults already block plain HTTP, and Android ignores user-installed certificates. Don't switch them off.
- Pinning stops a rogue or device-installed CA. It doesn't stop an attacker on their own phone.

- **Pin only if your team can maintain it for years:** pin the SPKI of an intermediate, ship a backup pin, and ship a new pin long before the new key goes live (§8.7).
- Never call `proceed()` on a WebView TLS error. It's the single worst network mistake.

Attestation (Chapters 9–10)

- A hacked app can't fake the result: Play Integrity's verdict reaches *your server* from Google, and your server checks an App Attest signature itself. That's why it beats an in-app root check.
- Play Integrity: check the package name, request hash or nonce, and timestamp first. Then the verdict.
- App Attest: attest once per key, assert on sensitive requests, and require a **strictly increasing counter** (the check people forget).
- Roll out in report-only mode first. Treat a verdict as a signal, not an on/off switch.

Biometrics (Chapter 11)

- A biometric success callback is a boolean an attacker can fake with Frida. **Bind the prompt to a key**, and have the server verify a signature made by that key.
- Android: Class 3 (`BIOMETRIC_STRONG`), `CryptoObject`. iOS: `.biometryCurrentSet`.
- New fingerprint added? The key is invalidated, and that is the protection working. On Android, letting the device PIN unlock the key, or letting one unlock cover later uses, turns this off.

Backend (Chapter 12)

- Never trust from the app: user ID, price, quantity, balance, permissions, limits.
- The one test for every endpoint: **if this request is replayed from another device, what stops it?**
- Rate-limit per account and per device, not per IP.

Resilience (Chapters 13–14)

- Anyone can decompile your app (JADX) and change it at runtime (Frida). Obfuscation buys time, not safety.
- Detect root, jailbreak, and hooking, **report** it, and let the server decide.

Pipeline (Chapter 15)

- Your CI can ship code under your signature, so attackers target it.
- Pin GitHub Actions to commit SHAs, give tokens no permissions by default (`permissions: {}`), and never put PR text into a `run:` step.
- Use Play App Signing, so a stolen upload key is recoverable. Put production releases behind a human approval.

WebViews (Chapter 16)

- A WebView runs web content *inside your app*. Load only your own `https` pages, check URLs on every navigation, turn off file access, and keep the JavaScript bridge small and checked.

IPC and deep links (Chapter 17)

- Export nothing by default. Use verified links (App Links, Universal Links), not `myapp://`, because any app can claim a custom scheme. Validate every link parameter.

Privacy (Chapter 18)

- Ask for less: the photo picker instead of storage access, approximate location instead of precise. Your SDKs' data collection is *your* privacy declaration.

Input (Chapter 19)

- Untrusted input is anything that crossed a boundary, including other apps, files, links, and AI output. Use parameterized SQL, and allowlists, not blocklists.

AI features (Chapter 20)

- Treat model output as untrusted. Require user confirmation before any action with side effects. Keep the model's API key on your server. A system prompt is not a secret.

Kotlin Multiplatform (Chapter 21)

- Share logic and validation. Keep keystores, biometrics, attestation, pinning enforcement, and WebViews native on each platform.

Testing and incidents (Chapters 22–24)

- Test your own app: `strings`, decompile, check storage, intercept traffic, hook with Frida, replay a request.
- Score with CVSS v4.0, which measures severity, not risk.
- Incident order: **contain, assess, notify, fix, learn.** Under GDPR you have 72 hours to notify the regulator.

---

4. Top interview questions, short answers

Foundations

- **Why not just obfuscate everything?** You can't prevent reverse engineering. Free tools decompile any app in minutes. Make secrets useless if found, and decide on the server.
- **What does "the client is not the trust boundary" mean?** The server makes every decision that matters; the app only gathers signals and proposes actions.
- **Root detection flags a device. What should the app do?** Report it to the backend and let the session continue. The backend can step up authentication, lower limits, or flag the account.
- **How should a mobile app sign users in?** Authorization code flow with PKCE (`S256`) and `state`, in the system browser, returning to a verified `https` link. Never a WebView login, never a client secret in the app.
- **What's the one rule for AES-GCM?** Never reuse a nonce with the same key.

Storage and keys

- **Where do you store a token on Android in 2026?** Access token in memory; refresh token encrypted with Tink under a Keystore key, saved through DataStore.
- **Is plain `SharedPreferences` insecure?** Mostly no. The sandbox and file-based encryption already protect it from other apps. Encrypt when your threat model includes a compromised device.
- **TEE vs StrongBox?** The TEE is a protected area of the main processor; StrongBox is a separate chip. Both stop the key being copied; StrongBox resists stronger hardware attacks.
- **Which Keychain class for a token?** `kSecAttrAccessibleWhenUnlockedThisDeviceOnly`: readable only while unlocked, and never usable on another device.
- **Where do secrets leak besides storage?** Logs, backups, app-switcher screenshots, the keyboard cache, notifications, and the clipboard.

Network

- **What does pinning protect against?** A wrong-but-trusted CA: a rogue CA, one installed on the device, or a state-mandated one.
- **Should you pin?** Only if your team can run it for years: backup pins, a rotation plan, and monitoring. Otherwise rely on strict TLS and Certificate Transparency.
- **Why pin the SPKI, not the certificate?** The certificate changes on every renewal; the public key can stay the same.
- **What breaks first when pinning goes wrong?** Every user on that version, at once, until a store update. That's why you ship backup pins.

Attestation

- **What does Play Integrity prove?** Your genuine app, from Play, on a device meeting an integrity level. It doesn't prove the user is honest.
- **Why is it better than an in-app root check?** The verdict comes from Google to your server, so a hacked app can't write its own.
- **What does App Attest check that people forget?** The counter must strictly increase, or a captured assertion can be replayed.
- **A returning user has a new App Attest key. Fraud?** Usually not. Reinstalls and new devices create new keys.

Biometrics

- **How do you make biometrics impossible to bypass?** Tie the prompt to a key operation, and have the server verify the result. A success callback alone is bypassed in minutes.
- **A thief adds their fingerprint. What stops them?** Keys that are invalidated when the enrolled fingerprints change.

Backend

- **What should never be trusted from the client?** Identity, prices, quantities, balances, permissions, and limits.

- **Bearer vs sender-constrained token?** A bearer token works for whoever holds it; a sender-constrained one (DPoP) works only with a key from the real device.

Pipeline

- **Why pin Actions to a commit SHA?** Anyone with push access can move a tag; nobody can change what a SHA points to.
- **A production key was committed months ago. First move?** Rotate it. Removing the commit doesn't help: it's already copied.

AI

- **What is indirect prompt injection?** Instructions hidden in content the model reads (a document, web page, or email) rather than typed by the user.
- **Can you keep a secret in the system prompt?** No. Anything in the model's context can be coaxed back out.

Judgment

- **Two weeks, one engineer: what first?** Find hardcoded secrets and rotate them. It removes the most risk per day.
- **PM wants "bank-level security". What do you ask?** What data, which regulator, and which threat: account takeover, fraud, data theft, or cloning?
- **Product wants to block rooted phones at launch.** Detect, report, and let the server decide. A local block is one hook away from removal and hurts real users.
- **A pentest says "no pinning, high severity".** It's a decision to make and record, not a bug to patch overnight. Pin if you can operate it; otherwise document why not.
- **"Is our app MASVS certified?"** No such certification exists. Offer a verification statement: which profile, which tests, which build, and when.

Every one of these, and 125 more, has a full answer in Part 10.

---

5. Myths vs facts

Myth	Fact
"Obfuscation hides our API key"	Anything in the app can be read. Move the key behind your server
"Root detection protects us"	It's one hook away from removal. Use it as a signal, not a gate
"HTTPS means nobody can read our traffic"	The device owner can: on iOS by trusting their own certificate, on Android by rooting or repackaging the app. HTTPS protects against *others* on the network
"Pinning makes us secure"	It stops one attack (a bad CA). Done badly, it causes outages
"Biometric success means the user is verified"	Only if the result is tied to a key the server checks
"Encrypt everything on the device"	Decide by threat model. Most leaks are logs, backups, and screenshots
"We're MASVS certified"	OWASP certifies nothing
"SLSA provenance means our build inputs are clean"	Provenance proves *how* it was built, not that the inputs were safe
"An on-device AI model fixes prompt injection"	It changes where data goes, not whether injection works

---

6. Numbers worth remembering

What	Number
Average data breach cost (IBM 2026)	$4.99M globally, $11.5M in the US, 247 days to identify and contain
MASVS / MASWE	24 controls in 8 categories / 78 weaknesses
GDPR breach notification	72 hours
EU Cyber Resilience Act	24-hour early warning, since 11 September 2026
Play Integrity default quota	10,000 token requests and 10,000 decryptions a day
App Attest	Under about 100 `attestKey` calls a second; ramp to at most 10 million users a day
TLS certificates	47 days maximum from 15 March 2029
Google Play target API	36

---

7. How to answer any design question

Use this order, out loud:

1. **What are we protecting?** Name the asset: tokens, payment data, personal data.
2. **Who is the attacker?** An opportunist, a cloner, a fraudster, or a targeted adversary. Each needs different controls (§1.3).
3. **Where is the trust boundary?** Usually between the app and your server. The server decides.
4. **What's the control, and how could it fail?** Prefer controls that fail independently, and say what happens when one does.
5. **How would you test it?** "I'd try to replay this request from another device" beats any list of features.

The answer that earns the most credibility: *"I attacked my own app, here's what I found, what I changed, and what I decided to accept."*

From Mobile Application Security: A Study Book for Engineers, CC BY-SA 4.0. Every line here is explained, sourced, and dated in the book.